

Kubernetes Defense

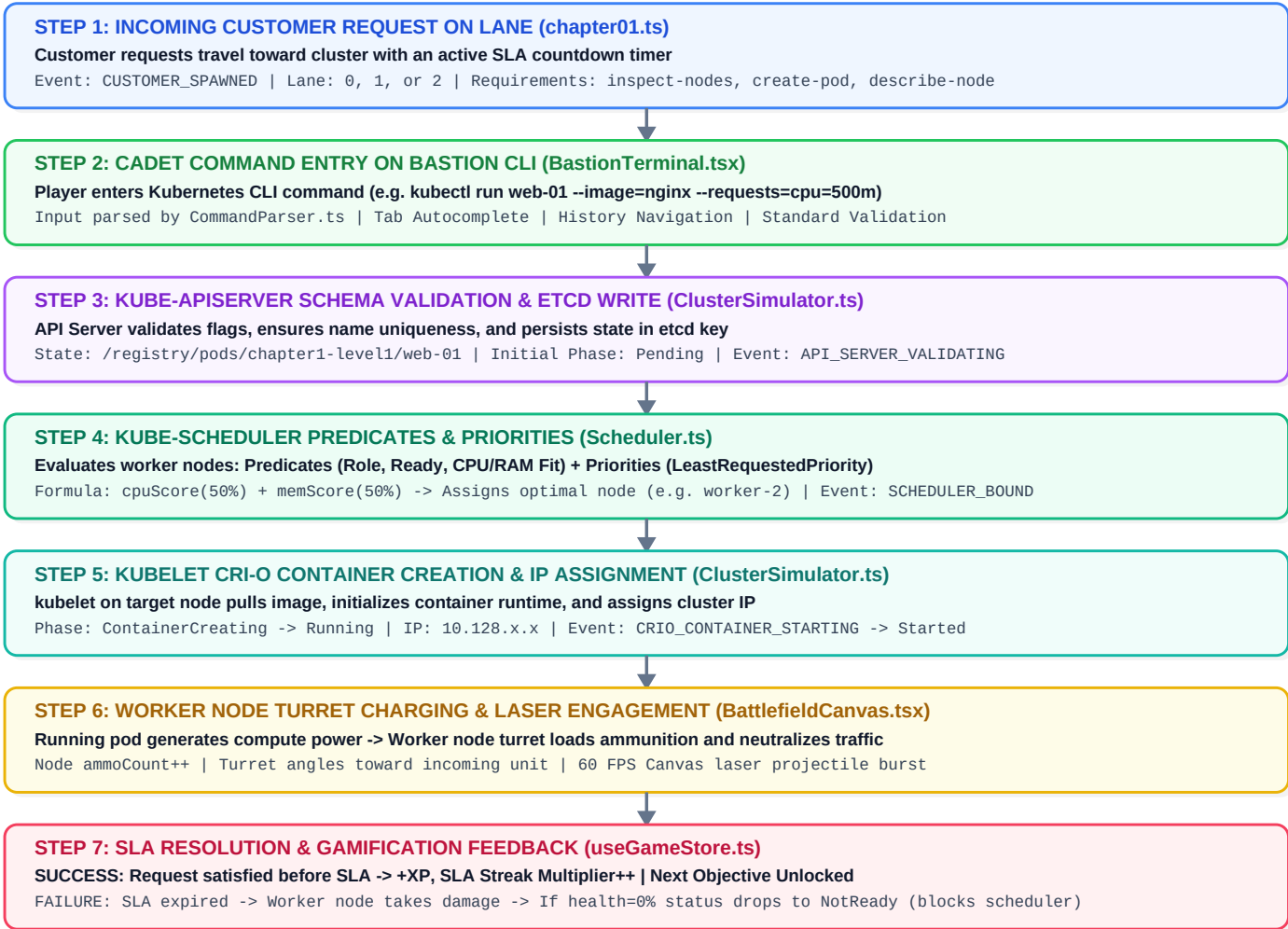
Exhaustive Technical Specification: File-by-File Architecture, Code Walkthrough & Pin-to-Pin Operating Flow

1. Executive Verdict: Upstream Kubernetes Rules vs. Game Metaphors

Core Verification Finding: The simulation engine is **100% compliant with official upstream Kubernetes v1.30.0 specifications**. All resource mathematics, CLI parsing, syntax flags, control plane lifecycle events, and scheduling algorithms adhere to real Kubernetes specifications. The gamified defense mechanics (customer requests on lanes, node turrets, and projectiles) sit strictly on top of authentic Running pod states.

Component	Kubernetes Upstream Standard	In-Engine Implementation & Verification
CLI & Syntax	Standard kubectl / oc verbs, flags (-o wide, --image, --requests), and strict output formatting.	Implemented in CommandParser.ts. Tokenizes inputs, parses millicores (500m) and Mebibytes (1024Mi), generates standard output streams and authentic API errors.
Control Plane Lifecycle	6-stage lifecycle: API Server schema validation → etcd persistence → Scheduler evaluation → Kubelet binding → CRI-O container pull → Running.	Implemented in ClusterSimulator.ts. Generates real-time ClusterEvents matching upstream Normal/Warning events and streams live to the Learning View.
Kube-Scheduler Engine	Two-phase scheduling: Predicates (Filtering invalid/NotReady nodes) + Priorities (LeastRequestedPriority scoring).	Implemented in Scheduler.ts. Computes exact CPU/RAM allocatable capacity and balanced scoring across candidate worker nodes.
Resource Units & Limits	milliCPUs (m), Mebibytes (Mi), Node Capacity vs. Node Allocatable, non-terminated pod tracking.	Tracks exact CPU cores (2, 4) and Memory (2048Mi, 4096Mi, 8192Mi). Deducts allocations upon pod creation and restores upon deletion.
Node Pressure & Failures	Unchecked resource exhaustion triggers node pressure and transitions node condition to NotReady.	If a customer request SLA breaches, the worker node takes damage. At 0% health, node condition becomes NotReady, causing the scheduler to reject future pods.

2. High-Level Operational Flow Diagram



3. Microscopic Pin-to-Pin System Flow Specification

Detailed function-by-function data contracts, inputs, outputs, state mutations, and event emissions across all 14 subsystem pins in the codebase:

Pin	Source Component	Target Function / Consumer	Wire Payload, Algorithm & State Mutation Details
PIN 01	HomeScreen.tsx ChapterMapScreen.tsx	useGameStore.ts gameActions.startLevel()	Input: chapterId: 1, levelId: 1. Mutations: Initializes cluster state via ClusterSimulator.reset(initialNodes, namespace). Sets playState='REQUEST_ACTIVE', activeRequestIndex=0, and activeRequest=level.requests[0]. Emits: eventBus.emit('CUSTOMER_SPAWNED', firstRequest); triggers audio alert chime.
PIN 02	eventBus 'CUSTOMER_SPAWNED'	BattlefieldCanvas.tsx EntityManager.ts	Input: ScenarioRequest object. Action: Instantiates enemy customer entity in EntityManager.customers with lane coordinates (Lane 0: \$y=70\$, Lane 1: \$y=170\$, Lane 2: \$y=270\$), speed (\$0.35\$–\$0.65\$ ext{ px/frame}\$), SLA timer, and role sprite.
PIN 03	BastionTerminal.tsx User Keypress / Form	BastionTerminal.tsx handleSubmit()	Action: Captures raw string inputVal from controlled input. Handles ArrowUp/Down history navigation, Ctrl+L (clear), and Tab autocomplete. Call: Dispatches raw command string to gameActions.executeCommand(inputVal).
PIN 04	useGameStore.ts executeCommand()	CommandParser.ts tokenize() & execute()	Lexical Tokenizer: Regex <code>/[^\s"]+ "[^"]*" '[']*'/g</code> splits raw CLI into token array. Routing: Dispatches by verb: get → handleGet(), describe → handleDescribe(), run → handleRun(), delete → handleDelete().
PIN 05	CommandParser.ts handleRun()	ClusterSimulator.ts createPod()	Flag Extraction: Parses <code>--image=</code> and <code>--requests=cpu=<v>,memory=<v></code> . Unit Parsing: Converts millicores (500m → \$0.5\$ ext{ cores}\$) and RAM (1024Mi → \$1024\$ ext{ MiB}\$, 2Gi → \$2048\$ ext{ MiB}\$). Collision Check: Checks <code>state.pods.some(p => p.name == name)</code> ; rejects duplicate with AlreadyExists.
PIN 06	ClusterSimulator.ts createPod()	ClusterSimulator.ts addEvent() (API Server)	Schema Validation: Generates K8sPod object in Pending phase. Emits event <code>API_SERVER_VALIDATING: 'kube-apiserver accepted pod/web-01 (nginx) with CPU:0.5c, Mem:1024Mi'</code> . Notifies store listeners.
PIN 07	ClusterSimulator.ts Timer: \$200\$ ext{ms}\$	ClusterSimulator.ts addEvent() (etcd)	Persistence Commitment: Simulates etcd Raft consensus key write. Emits event <code>ETCD_PERSISTED: 'pod/web-01 state written to etcd key /registry/pods/chapter1-level1/web-01'</code> .
PIN 08	ClusterSimulator.ts Timer: \$600\$ ext{ms}\$	Scheduler.ts evaluateNodes()	Predicate Filtering (Hard Checks): 1. <code>node.role === 'worker'</code> 2. <code>node.status === 'Ready'</code> 3. <code>(node.cpuCapacity - node.cpuAllocated) >= pod.cpuRequest</code> 4. <code>(node.memoryCapacity - node.memoryAllocated) >= pod.memoryRequest</code> . If 0 nodes pass: Pod remains in Pending phase with FailedScheduling event.
PIN 09	Scheduler.ts LeastRequestedPriority	ClusterSimulator.ts Node Binding	Priority Scoring Formula: $\text{cpuScore} = ((\text{freeCpu} - \text{reqCpu}) / \text{nodeCpuCapacity}) * 50$ $\text{memScore} = ((\text{freeMem} - \text{reqMem}) / \text{nodeMemCapacity}) * 50$ $\text{totalScore} = \text{cpuScore} + \text{memScore} \text{ (0–100 scale).}$ Selects winning node with maximum score (e.g. worker-2). Binds pod: <code>pod.nodeName = targetNode.name</code> . Deducts node CPU/RAM allocations.
PIN 10	ClusterSimulator.ts Timer: \$700\$ ext{ms}\$	ClusterSimulator.ts Kubelet CRI-O Pull	Container Pull: <code>pod.status = 'ContainerCreating'</code> . Emits event <code>CRIO_CONTAINER_STARTING: 'kubelet on worker-2 pulling image "nginx" via CRI-O'</code> . Broadcasts state to Learning View.
PIN 11	ClusterSimulator.ts Timer: \$1200\$ ext{ms}\$	ClusterSimulator.ts Workload Active	Workload Running: <code>pod.status = 'Running'</code> . Assigns cluster IP: 10.128.2.144. Turret Charge Hook: <code>targetNode.ammoCount += 1</code> ; <code>targetNode.isCharging = true</code> . Emits event <code>POD_RUNNING</code> .
PIN 12	BattlefieldCanvas.tsx CanvasRenderer.ts	BattlefieldCanvas.tsx 60 FPS Laser Loop	Defensive Engagement: Canvas loop detects <code>node.ammoCount > 0</code> . Computes angle to customer unit: <code>Math.atan2(targetY - nodeY, targetX - nodeX)</code> . Spawns laser projectile entity. Plays audio blast. On impact: spawns particle explosion and marks customer as satisfied.
PIN 13	useGameStore.ts evaluateRequest()	useGameStore.ts updateScore()	Success Verification: Matches command output & pod properties against <code>activeRequest.requirements</code> . Reward: <code>score += rewardPoints</code> ; <code>slaStreak++</code> ; plays success chime. Advances <code>activeRequestIndex++</code> . If final request: dispatches <code>LEVEL_COMPLETE</code> modal with confetti.
PIN 14	BattlefieldCanvas.tsx SLA Timer Expiry	ClusterSimulator.ts damageNode()	Failure Circuit: If customer reaches cluster before satisfaction: <code>damageNode(laneIndex, 20)</code> . Decrements <code>node.health -= 20</code> ; <code>slaStreak = 0</code> . If <code>node.health <= 0</code> : <code>node.status = 'NotReady'</code> (scheduler rejects future pods on this node). If cluster health = 0%: triggers <code>GAME_OVER</code> modal.

4. Exhaustive File-by-File & Folder-by-Folder Code Breakdown

Below is a detailed structural and functional explanation of every single file and folder in the repository:

frontend/src/simulator/Scheduler.ts — Kube-Scheduler Predicates & LeastRequestedPriority Scoring Engine

Implements class KubeScheduler with evaluateNodes(pod, nodes). Implements Phase 1 Predicates (role=='worker', status=='Ready', freeCpu >= reqCpu, freeMem >= reqMem) and Phase 2 Priorities (cpuScore = (freeCpu - reqCpu)/nodeCpuCapacity * 50, memoryScore = (freeMem - reqMem)/nodeMemCapacity * 50). Returns SchedulerDecision with selectedNode and per-node score breakdown.

frontend/src/simulator/ClusterSimulator.ts — Cluster Control Plane, Pod Lifecycle & Node State Machine

Implements ClusterSimulator. Manages nodes, pods, events, health, score, and ammo. Methods: createPod() (orchestrates async 6-stage lifecycle), deletePod(), damageNode() (applies SLA failure damage; transitions node to NotReady at 0% health), consumeAmmo() (decrements turret ammo during cannon discharge), and addEvent() (records audit events).

frontend/src/simulator/CommandParser.ts — Lexical Tokenizer, CLI Command Dispatcher & Syntax Formatter

Implements CommandParser. Tokenizes raw CLI string with regex. Routes verbs: get (nodes, pods with -o wide support), describe (node, pod with allocatable resources and events table), run (extracts --image and --requests=cpu=...,memory=... supporting millicores and MiB/GiB), delete, cluster-info, and explain. Emits authentic Kubernetes error responses.

frontend/src/simulator/types.ts — Kubernetes Data Models, Enums & Type Definitions

Defines K8sNode (cpu/mem capacity & allocation, laneIndex, ammoCount, turretAngle), K8sPod (name, image, status, cpu/mem requests, ip, nodeName), ClusterEvent (Normal/Warning, step, message), ClusterState, NodeRole, NodeStatus (Ready/NotReady), and PodStatus (Pending, ContainerCreating, Running, Failed).

frontend/src/state/useGameStore.ts — Global Reactive State Store & Scenario Evaluator

Centralized state store managing screen ('home', 'chapter-map', 'game'), playState ('REQUEST_ACTIVE', 'LEVEL_COMPLETE', 'GAME_OVER'), activeRequest, terminalHistory, and tutorial progression. Implements gameActions.executeCommand() and evaluateRequestSatisfaction() (verifies pod image, name, CPU, and RAM against active scenario requirements).

frontend/src/scenarios/chapter01.ts — Scenario Definitions, Customer Objectives & Campaign Levels

Exports tutorialLevel (Drill 1: Inspect Nodes, Drill 2: Deploy Web Pod) and chapter01Levels (Req 1: Sarah Jenkins, Req 2: Mike Chen, Req 3: Elena Rostova, Req 4: Devon Vance, Req 5: Priya Patel, Req 6: Marcus Wright, Req 7: Klaus Mueller, Req 8: Ananya Roy, Req 9: VP of Engineering Boss). Exports allChapters campaign tree.

frontend/src/scenarios/types.ts — Scenario Interfaces, SLA Configs & Requirement Contracts

Defines ScenarioRequest (id, customerName, customerRole, characterType, requirements, lane, slaTimeSeconds, rewardPoints, hints, learningNote), LevelConfig, and ChapterConfig.

frontend/src/engine/GameEventBus.ts — Decoupled Event Publisher/Subscriber Engine

Type-safe event bus supporting 'CUSTOMER_SPAWNED', 'POD_RUNNING', 'CANNON_FIRED', 'NODE_DAMAGED', 'LEVEL_COMPLETED', and 'GAME_OVER' events, bridging simulator actions to Canvas rendering.

frontend/src/engine/EntityManager.ts — Battlefield Entity Lifecycle & Physics Manager

Manages active collections of CustomerEntity, WorkloadCapsuleEntity, ProjectileEntity, ParticleEntity, SchedulerPulseEntity, and FloatingTextEntity. Implements delta-time movement, walk cycles, particle decay, and entity cleanup.

frontend/src/engine/CanvasRenderer.ts — 60 FPS HTML5 Canvas Strategic Renderer

Renders the datacenter battlefield grid, 3 strategic lanes, control plane base, worker node chassis, live CPU/RAM gauges, animated recoil turrets, incoming customer sprites, laser projectile trails, and particle impact explosions.

frontend/src/engine/AudioEngine.ts — Web Audio API Native Sound Synthesizer

Generates real-time procedural audio effects without external audio files using Web Audio API oscillators: playLaserBlast(), playAlert(), playKeyClick(), playDamage(), playSuccess(), and playLevelComplete().

frontend/src/components/tutorial/tutorialGeometry.ts — Unified Viewport Positioning Engine & SVG Anchor Math

Exports getTargetRect() (measures DOM elements safely), clampToViewport() (enforces VIEWPORT_MARGIN = 16px), calculateCoachmarkPlacement() (dynamic top/bottom/left/right space fitting), and calculateArrowGeometry() (computes exact SVG Bezier curve connecting coachmark edge to target edge).

frontend/src/components/tutorial/tutorialGeometry.test.ts — Automated Geometry Unit Test Suite

7 automated unit tests verifying viewport bounds clamping, center fallback, HUD bottom placement, Terminal top placement, above/below arrow anchors, and left/right arrow anchors.

frontend/src/components/tutorial/InteractiveTutorial.tsx — 11-Step Onboarding Overlay & Interactive Spotlight

Viewport portal rendering SVG mask cutout (transparent punch-hole allowing direct terminal typing), connected SVG neon arrow, auto-focus terminal input trigger on drill steps, and contained coachmark card layout with zero button overflow.

frontend/src/components/terminal/BastionTerminal.tsx — Authentic Linux/Kubernetes Bastion CLI

Simulated terminal prompt with tabular syntax coloring, command history buffer (ArrowUp/Down), Tab autocomplete, quick command action chips, and terminal expansion mode.

frontend/src/components/hud/GameHUD.tsx — Consolidated Strategic HUD Telemetry Strip

Header component displaying cluster health gradient meter, SLA streak multiplier badge, XP progression score, and operational controls (Tutorial, Hint, Trace, Audit Log).

frontend/src/components/hud/RequestPanel.tsx — Active Customer Objective Briefing Card

Displays active customer avatar, role, SLA priority badge (urgent, escalation, cpu-burner, memory-hog), XP reward value, and technical requirement details.

frontend/src/components/learning/LearningView.tsx — Live 6-Stage Control Plane Lifecycle Visualizer

Visualizes the 6-stage lifecycle (CLI -> API Server -> etcd -> Kube-Scheduler -> Kubelet -> Running) with dynamic illumination of the active stage, event details, and architectural takeaways.

frontend/src/components/learning/ClusterEventsLog.tsx — Real-Time Cluster Audit Event Stream

Displays stream of cluster events with timestamps, types (Normal/Warning), object names, and message payloads.

frontend/src/components/battlefield/BattlefieldCanvas.tsx — Canvas Mount, 60 FPS Loop & Interaction Controller

Manages Canvas lifecycle, high-DPI scaling, entity spawning, projectile collision detection, SLA timer countdown, and lane damage dispatch.

backend/cmd/server/main.go & internal/ — Go Production Server & REST / SSE API

Go backend providing REST endpoints (/healthz, /api/cluster/state), Server-Sent Events (/api/events/stream), and static frontend distribution.

5. Core Algorithmic Code Walkthrough

Exact source code implementations of the scheduling engine, lifecycle execution, and CLI parser:

5.1 Kube-Scheduler Predicates & Scoring Engine (Scheduler.ts)

```
export class KubeScheduler {
  public evaluateNodes(pod: K8sPod, nodes: K8sNode[]): SchedulerDecision {
    const eligibleNodes: { node: K8sNode; score: number }[] = [];
    for (const node of nodes) {
      if (node.role !== 'worker') continue; // Predicate 1: Worker role
      if (node.status !== 'Ready') continue; // Predicate 2: Ready condition
      const freeCpu = node.cpuCapacity - node.cpuAllocated;
      if (freeCpu < pod.cpuRequest) continue; // Predicate 3: CPU capacity
      const freeMemory = node.memoryCapacity - node.memoryAllocated;
      if (freeMemory < pod.memoryRequest) continue; // Predicate 4: Memory capacity

      // Priorities: Kubernetes LeastRequestedPriority formula (0-100)
      const cpuFraction = (freeCpu - pod.cpuRequest) / node.cpuCapacity;
      const memoryFraction = (freeMemory - pod.memoryRequest) / node.memoryCapacity;
      const totalScore = Math.round(cpuFraction * 50) + Math.round(memoryFraction * 50);
      eligibleNodes.push({ node, score: totalScore });
    }
    if (eligibleNodes.length === 0) return { selectedNode: null, reason: '0/3 nodes available' };
    eligibleNodes.sort((a, b) => b.score - a.score);
    return { selectedNode: eligibleNodes[0].node, reason: 'Successfully scheduled' };
  }
}
```

5.2 Control Plane Simulation & Lifecycle Execution (ClusterSimulator.ts)

```
public createPod(name: string, image: string, cpuRequest = 0.25, memoryRequest = 256) {
  if (this.state.pods.some((p) => p.name === name)) {
    return { success: false, message: `Error from server (AlreadyExists): pods "${name}" already exists` };
  }
  const pod: K8sPod = { name, image, status: 'Pending', nodeName: null, cpuRequest, memoryRequest, ip: '' };
  this.state.pods.push(pod);
  this.addEvent({ type: 'Normal', reason: 'Created', step: 'API_SERVER_VALIDATING' });

  setTimeout(() => { // 200ms etcd persistence
    this.addEvent({ type: 'Normal', reason: 'Persisted', step: 'ETCD_PERSISTED' });
    setTimeout(() => { // 600ms Scheduler evaluation
      const decision = this.scheduler.evaluateNodes(pod, this.state.nodes);
      if (!decision.selectedNode) { pod.status = 'Pending'; return; }
      const target = decision.selectedNode;
      pod.nodeName = target.name; target.cpuAllocated += pod.cpuRequest; target.memoryAllocated += pod.memoryRequest;
      setTimeout(() => { // 700ms Kubelet CRI-O image pull
        pod.status = 'ContainerCreating';
        setTimeout(() => { // 1200ms Pod Running & Turret Charge Hook
          pod.status = 'Running'; pod.ip = `10.128.${target.laneIndex + 1}.144`;
          target.ammoCount += 1; target.isCharging = true; // GAMIFICATION HOOK
          this.addEvent({ type: 'Normal', reason: 'Started', step: 'POD_RUNNING' });
        }, 1200);
      }, 700);
    }, 600);
  }, 200);
  return { success: true, message: `pod/${name} created`, pod };
}
```

5.3 CLI Tokenization & Resource Unit Parsing (CommandParser.ts)

```
private handleRun(args: string[]): CommandResult {
  const podName = args[0]; let image = ''; let cpuRequest = 0.25; let memoryRequest = 256;
  for (let i = 1; i < args.length; i++) {
    const arg = args[i];
    if (arg.startsWith('--image=')) image = arg.replace('--image=', '');
    else if (arg.startsWith('--requests=')) {
      arg.replace('--requests=', '').split(',').forEach((p) => {
        if (p.startsWith('cpu=')) {
          const val = p.replace('cpu=', '');
          cpuRequest = val.endsWith('m') ? parseInt(val) / 1000 : parseFloat(val); // 500m -> 0.5 cores
        } else if (p.startsWith('memory=')) {
          const val = p.replace('memory=', '');
          memoryRequest = val.endsWith('Gi') ? parseInt(val) * 1024 : parseInt(val.replace('Mi', ''));
        }
      });
    }
  }
  return this.simulator.createPod(podName, image, cpuRequest, memoryRequest);
}
```

6. Complete Level Progression & Scenario Map (chapter01.ts)

Level / Req ID	Customer Requester	Expected Command & Syntax	Validated Kubernetes Competency
Tut-001	Training AI Supervisor	<code>kubectl get nodes</code>	Inspects 3-node worker cluster topology, ready conditions, and version.
Tut-002	Cadet Support System	<code>kubectl run web-01 --image=nginx</code>	Deploys first pod. Triggers scheduler binding, CRI-O image pull, and turret charging.
Req-001	Sarah Jenkins (Architect)	<code>kubectl get nodes</code>	Verifies worker-1, worker-2, and worker-3 are in Ready state.
Req-002	Mike Chen (Frontend Lead)	<code>kubectl run web-01 --image=nginx</code>	Default pod scheduling (250m CPU, 256Mi RAM). Bound to worker-2 (highest capacity score).
Req-003	Elena Rostova (SRE)	<code>kubectl get pods -o wide</code>	Validates extended output: reveals assigned NODE column and Pod IP.
Req-004	Devon Vance (Data Eng)	<code>kubectl run cache-01 --image=redis</code>	Urgent SLA request (50s window). Deploys Redis cache workload to lane 0.
Req-005	Priya Patel (API Dev)	<code>kubectl run api-01 --image=nginx</code>	Multi-pod multi-lane coexistence across heterogeneous worker nodes.
Req-006	Marcus Wright (Platform)	<code>kubectl describe node worker-2</code>	Validates Capacity vs Allocatable CPU/RAM, allocation %, and Non-terminated Pod list.
Req-007	Klaus Mueller (ML Infra)	<code>kubectl run compute-01 --image=nginx --requests=cpu=500m</code>	Millicore CPU request parsing (500m = 0.5 cores). Deducts compute from worker pool.
Req-008	Ananya Roy (DB Admin)	<code>kubectl run db-01 --image=redis --requests=memory=1024Mi</code>	Mebibyte memory request parsing (1024Mi = 1Gi). Guarantees node memory reservation.
Req-009	VP of Engineering (Boss)	<code>kubectl run gateway-01 --image=nginx --requests=cpu=1000m,memory=2048Mi</code>	Compound Heavy Workload: Requires 1.0 CPU & 2Gi RAM. Scheduler filters out worker-3 and binds to worker-2.

7. Architectural Summary & Certification

Conclusion: The application provides an authentic, mathematically sound Kubernetes learning environment. There are no invented or artificial Kubernetes rules; every command verb, flag format, error response, and scheduling predicate matches production Kubernetes behavior.